Divide and Conquer

1. Divide and Conquer
   - obtaining subquadratic time
   - measuring disorder by counting inversions
   - a sort and count algorithm
2. Finding the Closest Pair of Points
   - distances between pairs of points in the plane
   - combining solutions of subproblems
3. Karatsuba's Integer Multiplication
   - improving the elementary school algorithm
   - a recursive integer multiplication

CS 401/MCS 401 Lecture 8
Computer Algorithms I
Jan Verschelde, 6 July 2026

Divide and Conquer

divide and conquer

Consider the following type of algorithm:

1. break the input into several, smaller parts;
2. solve the problem on each part recursively; and
3. combine the solutions to the subproblems into an overall solution.

Mergesort splits a list of numbers in two halves,
sorts the halves, and merges the sorted halves.
For an input of n numbers, mergesort runs in $O(n \log(n))$.

Straightforward algorithms for the problems we consider run in $O(n^2)$.

With divide and conquer we can reduce the $O(n^2)$ to $O(n \log(n))$ or to *subquadratic time*, that is: $O(n^p)$, with $1 < p < 2$.

Divide and Conquer

measuring disorder by counting inversions

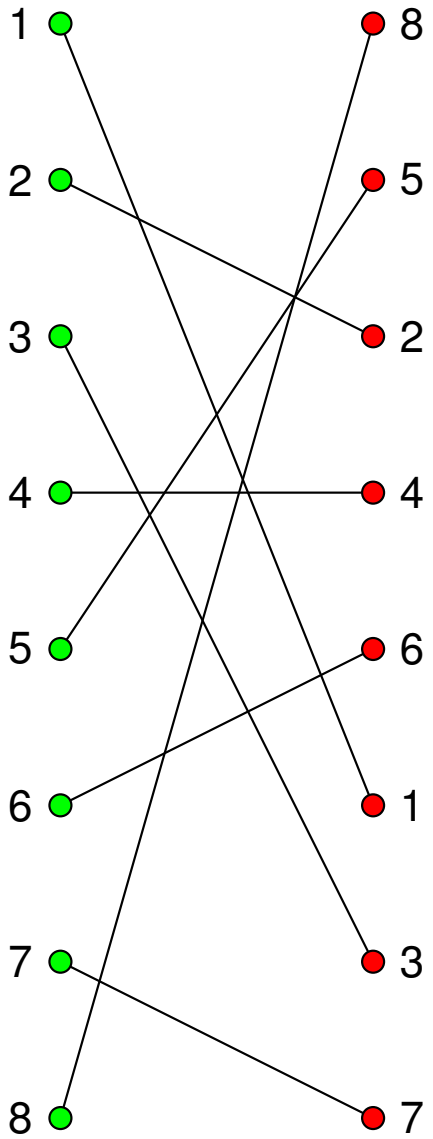
Definition (1)

Consider a sequence of numbers: $a_1, a_2, \ldots, a_n$. Two indices $i < j$ form *an inversion* if $a_i > a_j$.

Inversions in 8, 5, 2, 4, 6, 1, 3, 7 are

- (8, 5), (8, 2), (8, 4), (8, 6), (8, 1), (8, 3), (8, 7);
- (5, 2), (5, 4), (5, 1), (5, 3);
- (2, 1);
- (4, 1), (4, 3);
- (6, 1), (6, 3).

Count the crossing line segments in the picture at the right.

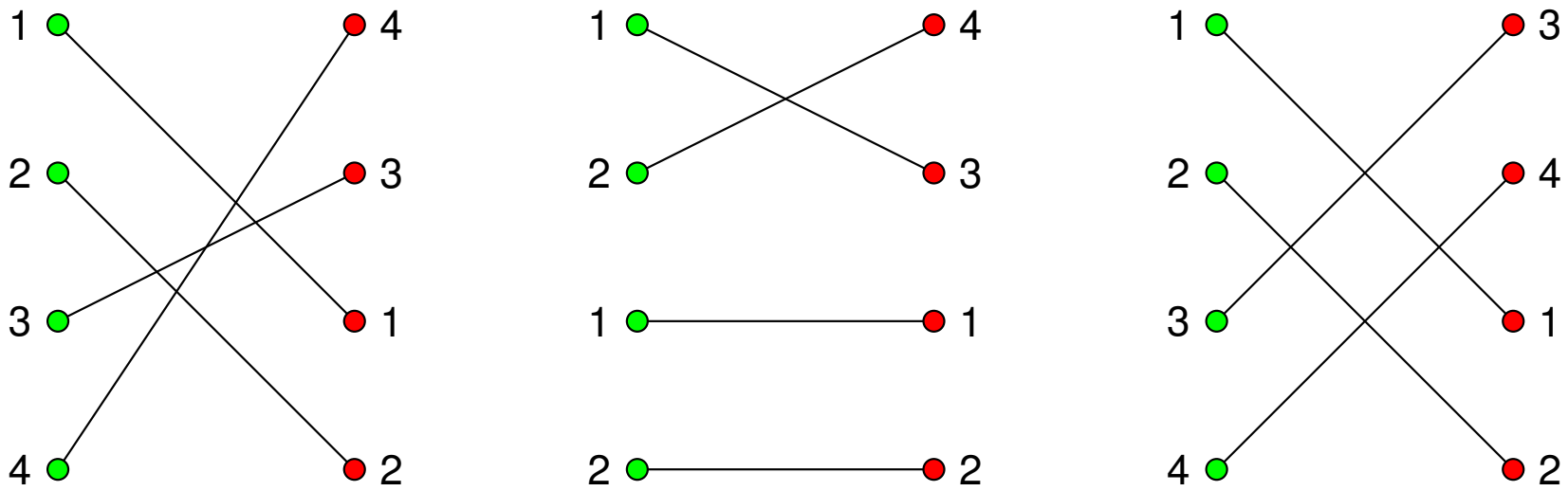


counting inversions by divide and conquer

A straightforward count compares i and j of every pair (a_i, a_j), which obviously takes $O(n^2)$ time for n numbers.

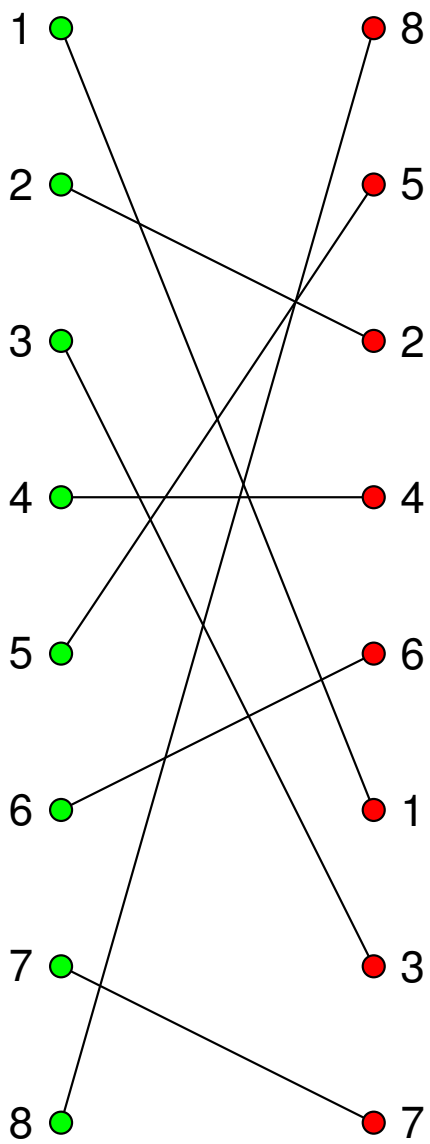
Applying divide and conquer to count inversions:

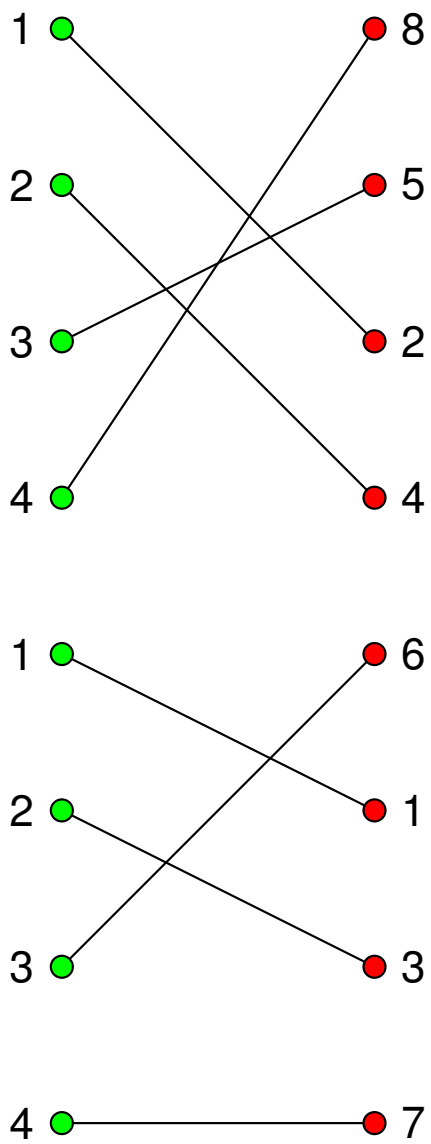
1. count the number of inversions of two halves, and
2. count the inversions of number in two diferent halves.

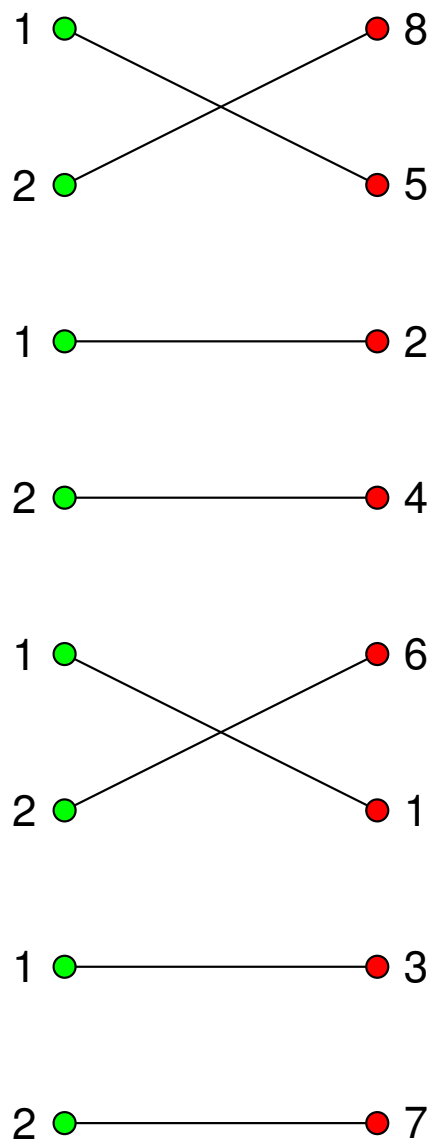


Split $4, 3, 1, 2$ in $4, 3$ and $1, 2$: one inversion in $4, 3$.
Four inversions between the halves. Total: 5 inversions.

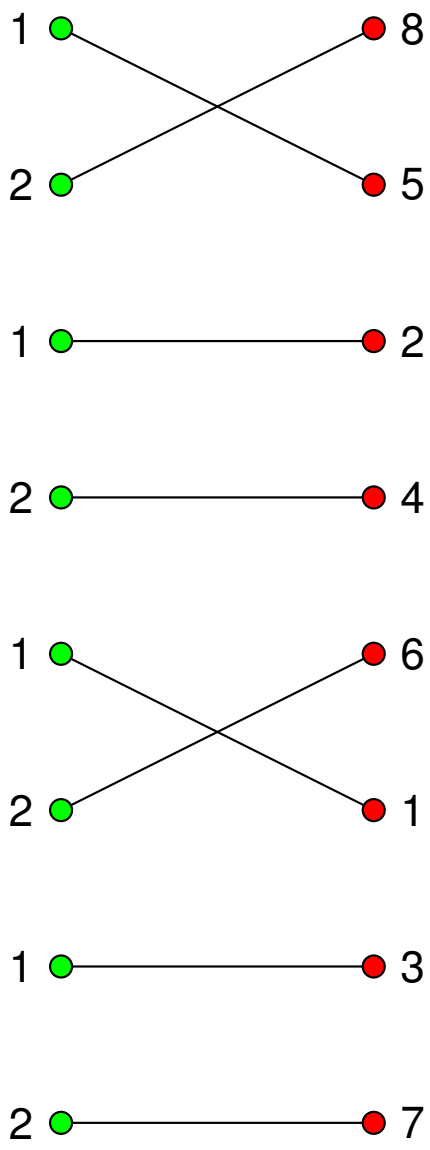
counting inversions by dividing in half: 2 at base

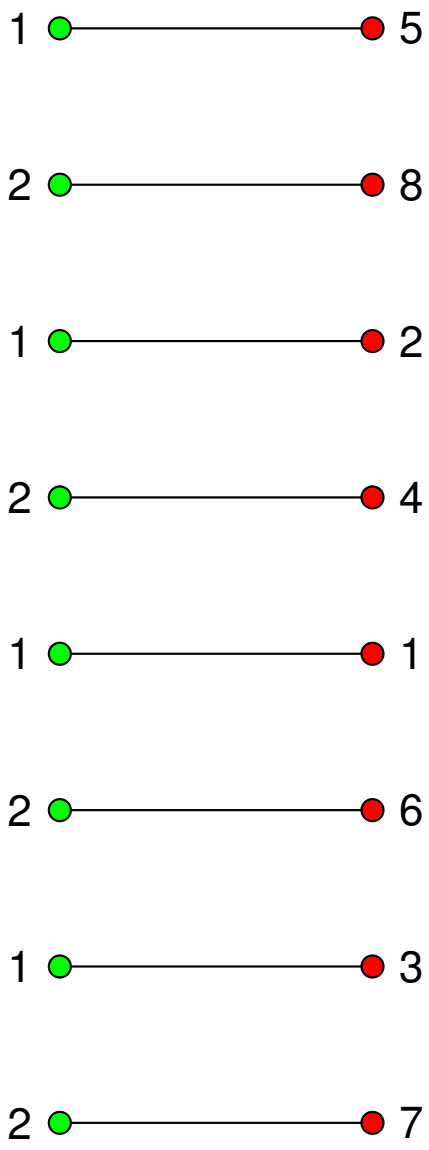


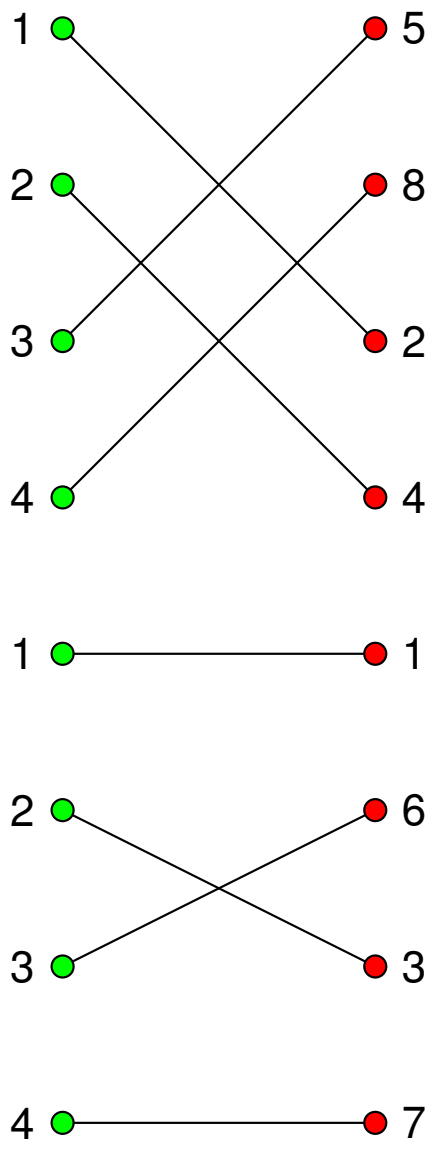




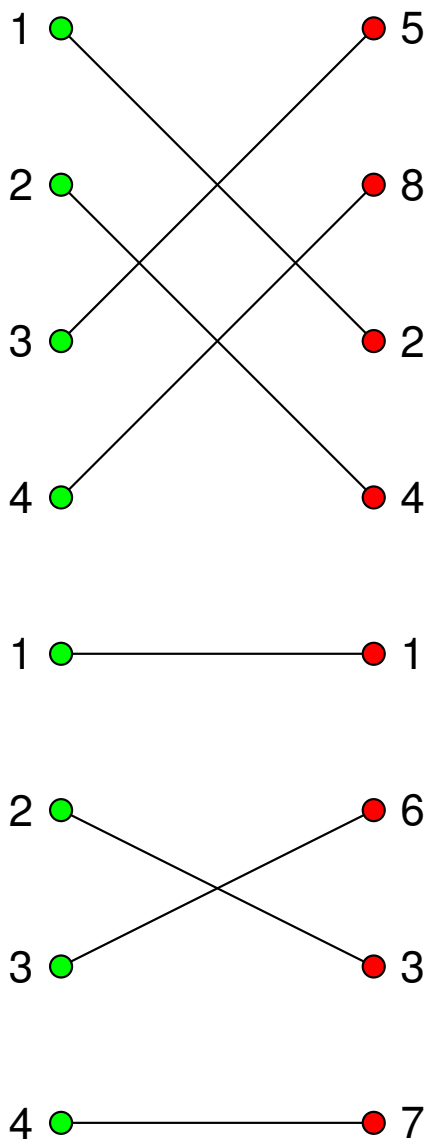
sorting & merging the halves: $2 + 4 + 1 = 7$

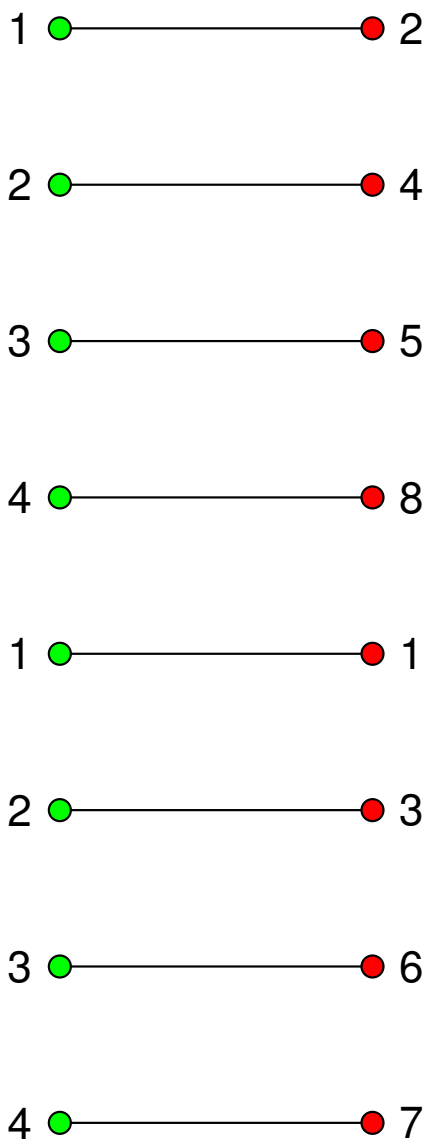


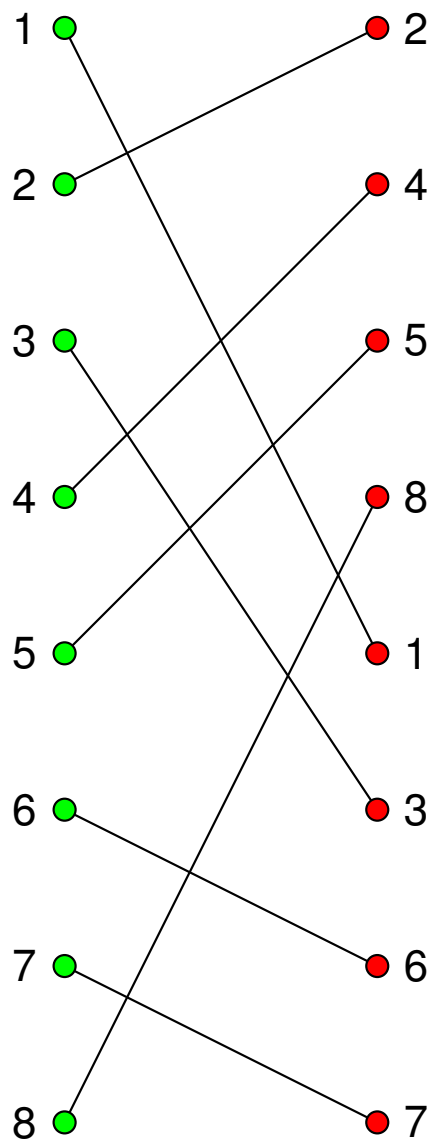




sorting & merging the halves: $7 + 9 = 16$







a first exercise

Exercise 1:

Consider as given n line segments between points with coordinates $(0, i)$ and $(1, a_i)$, for $i = 0, 1, \ldots, n-1$,
where $a_i \in \{ 0, 1, \ldots, n-1 \}$ and $a_i \neq a_j$ for $i \neq j$.

Show that the number of crossings between the line segments equals the number of inversions in the sequence $a_0, a_1, \ldots, a_{n-1}$.

Divide and Conquer

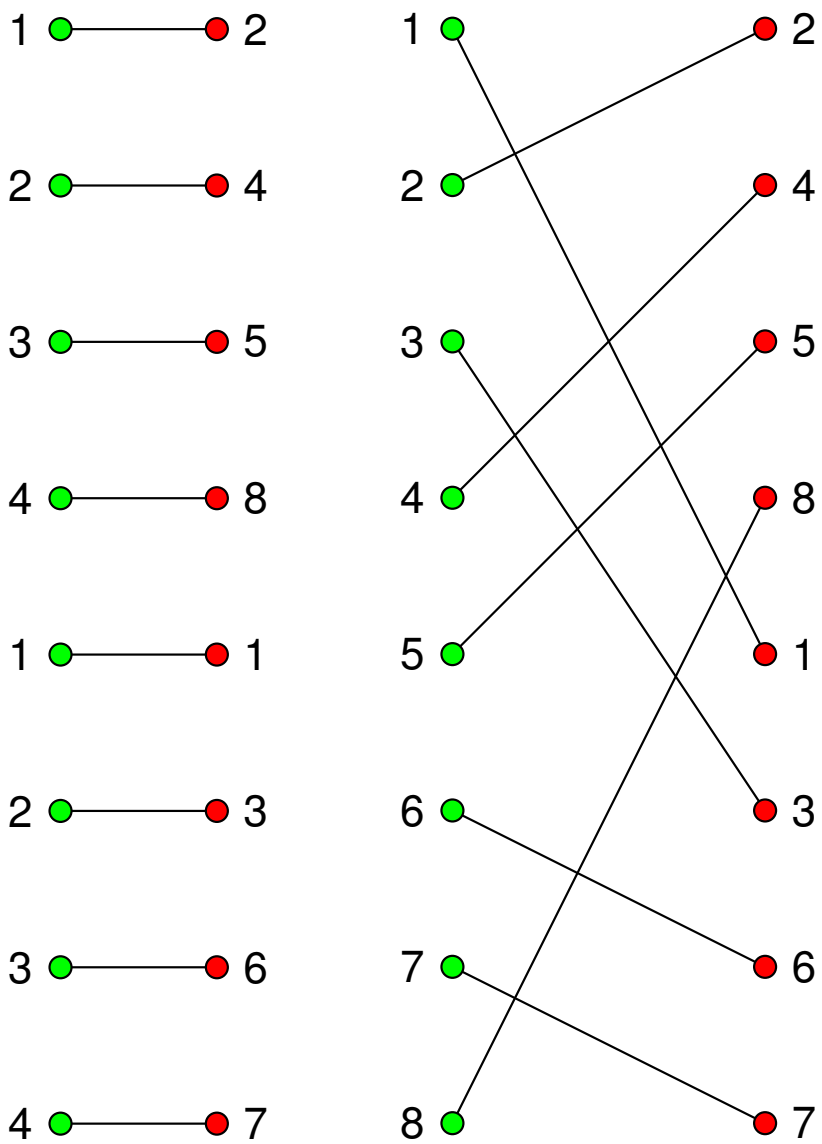
a sort and count algorithm

`Sort-and-Count`(L)

Input: L, a list of numbers.
Output: r_L, the number of inversions in L; and
S_L, the sorted list of numbers in L.

$n := \#L$
if $n \leq 1$ then
  return $(0, L)$
else
  let A contain the first $\lceil n/2 \rceil$ numbers of L
  let B contain the remaining $\lfloor n/2 \rfloor$ numbers of L
  $(r_A, S_A) :=$ `Sort-and-Count`(A)
  $(r_B, S_B) :=$ `Sort-and-Count`(B)
  $(r, S_L) :=$ `Merge-and-Count`(S_A, S_B)
  return $(r_A + r_B + r, S_L)$

merging and counting



$r := 0$

- Merge

$A = 2, 4, 5, 8$, $B = 1, 3, 6, 7$.
As $1 \in B$ is smallest, 1 makes an inversion with all elements in A. $r := r + \#A = 4$.
Pop 1 from B, 2 from A.

- Merge

$A = 4, 5, 8$, $B = 3, 6, 7$.
As $3 \in B$ is smallest 3 makes an inversion with all elements in A. $r := r + \#A = 4 + 3 = 7$.
Pop 3 from B, 4, 5 from A.

- Merge $A = 8$, $B = 6, 7$.

Both 6 and 7 make one inversion. $r := r + \#A = 7 + 1 = 8$, $r := r + \#A = 8 + 1 = 9$.

a merge and count algorithm

`Merge-and-Count`(A, B)

Input: A, a sorted list of numbers; and
B, a sorted list of numbers.
Output: r_L, the number of inversions between A and B;
S_L, the sorted list of numbers in A and B.

$r := 0$; $S := \emptyset$
while $A \neq \emptyset$ and $B \neq \emptyset$ do
  if $\texttt{top}(A) \leq \texttt{top}(B)$ then
    $a := \texttt{pop}(A)$; $\texttt{append}(S, a)$
  else
    $b := \texttt{pop}(B)$; $\texttt{append}(S, b)$; $r := r + \#A$
if $A = \emptyset$ then $\texttt{append}(S, b)$ for all $b \in B$
if $B = \emptyset$ then $\texttt{append}(S, a)$ for all $a \in A$
return (r, S)

an observation on the correctness

Theorem (2)

`Sort-and-Count` *counts correctly the inversions.*

As `Merge-and-Count` does the count, it is implied by Lemma (3).

Lemma (3)

`Merge-and-Count` *counts correctly the inversions.*

Observe: if an element of the second set is popped, then it makes an inversion of all elements of the first set because all smaller elements have already been popped.

Therefore, $r := r + \#A$ counts correctly.

analysis of the running time

The running time of `Merge-and-Count`(A, B) is $O(n)$, $n = \#A + \#B$.

Let $T(n)$ be the time for `Sort-and-Count`(L), $n = \#L$.

$$T(n) \leq 2T(n/2) + cn,$$

where c is the constant in the $O(n)$ of `Merge-and-Count`.

Theorem (4)

If $T(n) \leq 2T(n/2) + c_1 n$ and $T(2) \leq c_2$,
for positive constants c_1 and c_2, then $T(n)$ is $O(n \log(n))$.

solving a recurrence by substitution

Theorem (5)

If $T(n) \leq 2T(n/2) + c_1 n$ and $T(2) \leq c_2$,
for positive constants c_1 and c_2, then $T(n)$ is $O(n \log(n))$.

Proof. We proceed by induction on n.
The base case is $n = 2$, $T(2)$ is $O(1)$ and $O(2\log(2))$ is $O(1)$.
Induction hypothesis: $T(k)$ is $O(k \log(k))$ for all $k < n$.
There are constants c_3 and k_0: $T(k) \leq c_3 k \log(k)$, for $k \geq k_0$.

$$\begin{aligned}
T(n) &\leq 2T(n/2) + c_1 n \\
&\leq 2c_3(n/2)\log(n/2) + c_1 n, \text{ for } n \geq n_0 \\
&\leq 2c_3(n/2)(\log(n) - \log(2)) + c_1 n, \text{ for } n \geq n_0 \\
&\leq c_3 n(\log(n) - 1) + c_1 n, \text{ for } n \geq n_0 \\
&\leq c_3 n\log(n) - c_3 n + c_1 n, \text{ for } n \geq n_0 \\
&\leq c_3 n\log(n), \text{ for } n \geq n_0
\end{aligned}$$

Thus $T(n)$ is $O(n \log(n))$. Q.E.D.

ceil and floor

Do $\lceil \cdot \rceil$ and $\lfloor \cdot \rfloor$ matter to upper bounds?

Exercise 2:

Consider the recurrence $T(n) \leq T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + n$.

Show that $T(n)$ is $O(n \log(n))$.

a modification of the problem

Exercise 3:
Assume the numbers on input are the centers of circles with equal radius r, all on the same x-axis, so only their x-coordinate is different.

Consider the modified definition of an inversion:

Definition (6)

Consider a sequence of numbers: $a_1, a_2, \ldots, a_n$.
For $r > 0$, two indices $i < j$ form *an inversion* if $a_i - a_j > r$.

Write an $O(n \log(n))$ algorithm to count the number of inversions.
Prove that its running time is $O(n \log(n))$, independent of the constant r.

Divide and Conquer

1. Divide and Conquer
   - obtaining subquadratic time
   - measuring disorder by counting inversions
   - a sort and count algorithm
2. Finding the Closest Pair of Points
   - distances between pairs of points in the plane
   - combining solutions of subproblems
3. Karatsuba's Integer Multiplication
   - improving the elementary school algorithm
   - a recursive integer multiplication

CS 401/MCS 401 Lecture 8
Computer Algorithms I
Jan Verschelde, 6 July 2026

Divide and Conquer

1. Divide and Conquer
   - obtaining subquadratic time
   - measuring disorder by counting inversions
   - a sort and count algorithm
2. Finding the Closest Pair of Points
   - distances between pairs of points in the plane
   - combining solutions of subproblems
3. Karatsuba's Integer Multiplication
   - improving the elementary school algorithm
   - a recursive integer multiplication

computing the minimum distance between two points

Given: $P = \{ (p_x, p_y) \mid p_x, p_y \in \mathbb{R} \}$ points in the plane, $n = \#P$.

Output: $d = \min_{(p_x,p_y),(q_x,q_y) \in P} \sqrt{(p_x - q_x)^2 + (p_y - q_y)^2}$.

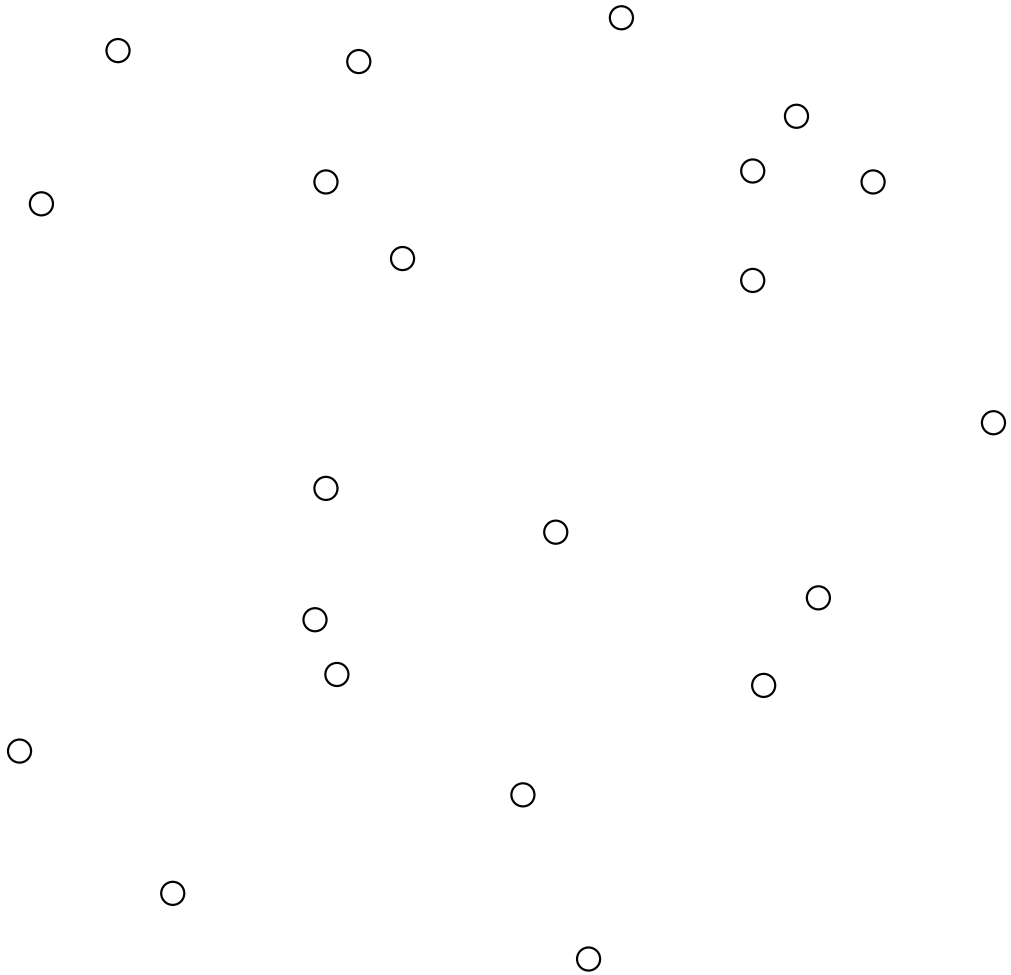
$d_{\min} := \infty$
for $(p_x, p_y) \in P$ do
  for $(q_x, q_y) \in P$, $(q_x, q_y) \neq (p_x, p_y)$ do
    $d_{\min} := \min(d_{\min}, \sqrt{(p_x - p_x)^2 + (q_y - q_y)^2})$.

Notice the nested, double loop:

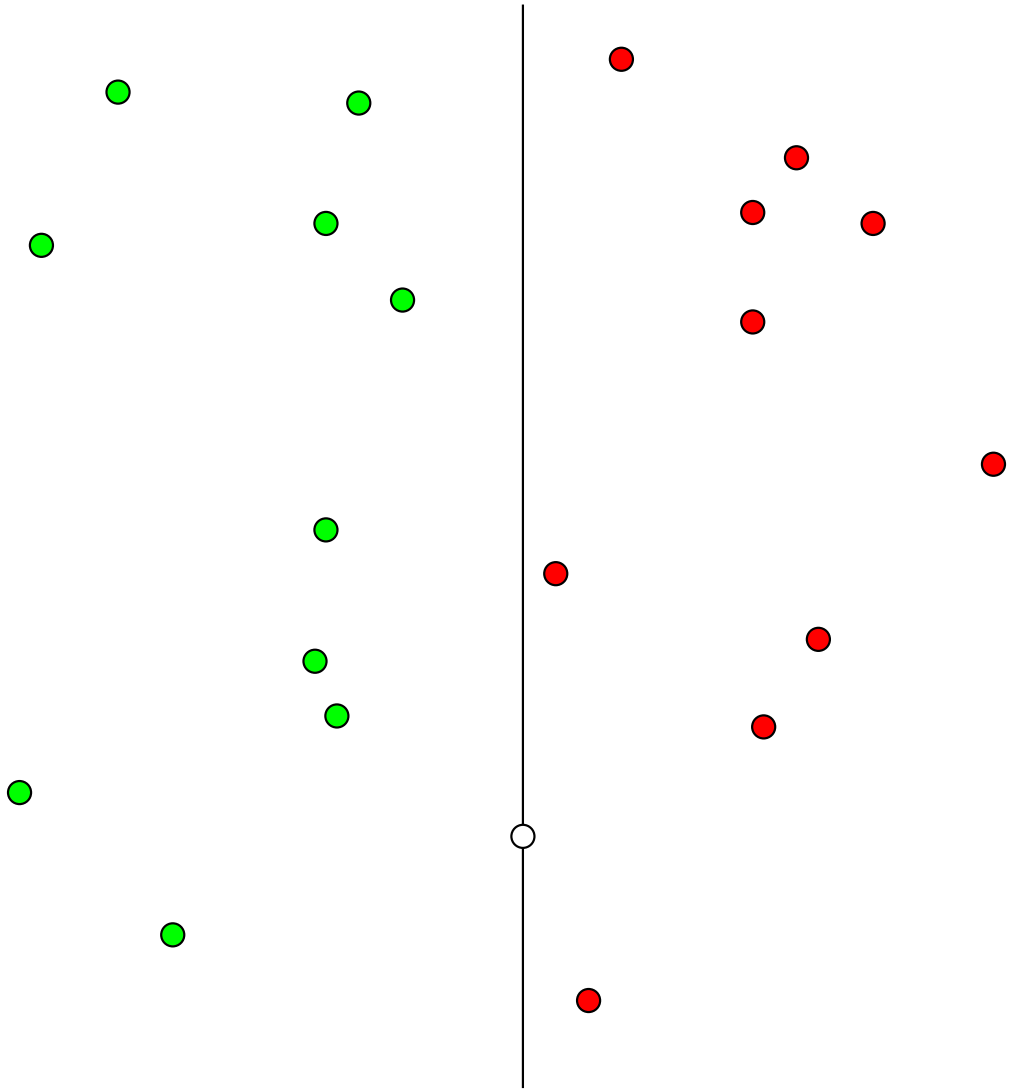
1. The first loop is executed n times.
2. For the second loop we have $n - 1$ choices for (q_x, q_y).

So we have $n(n-1)$ steps. Observe that the distance for (p_x, p_y), (q_x, q_y) is the same as the distance for $(q_x, q_y), (p_x, p_y)$, so solving this problem could be done in $\frac{1}{2}n(n-1)$ steps, twice as fast, but still $O(n^2)$.

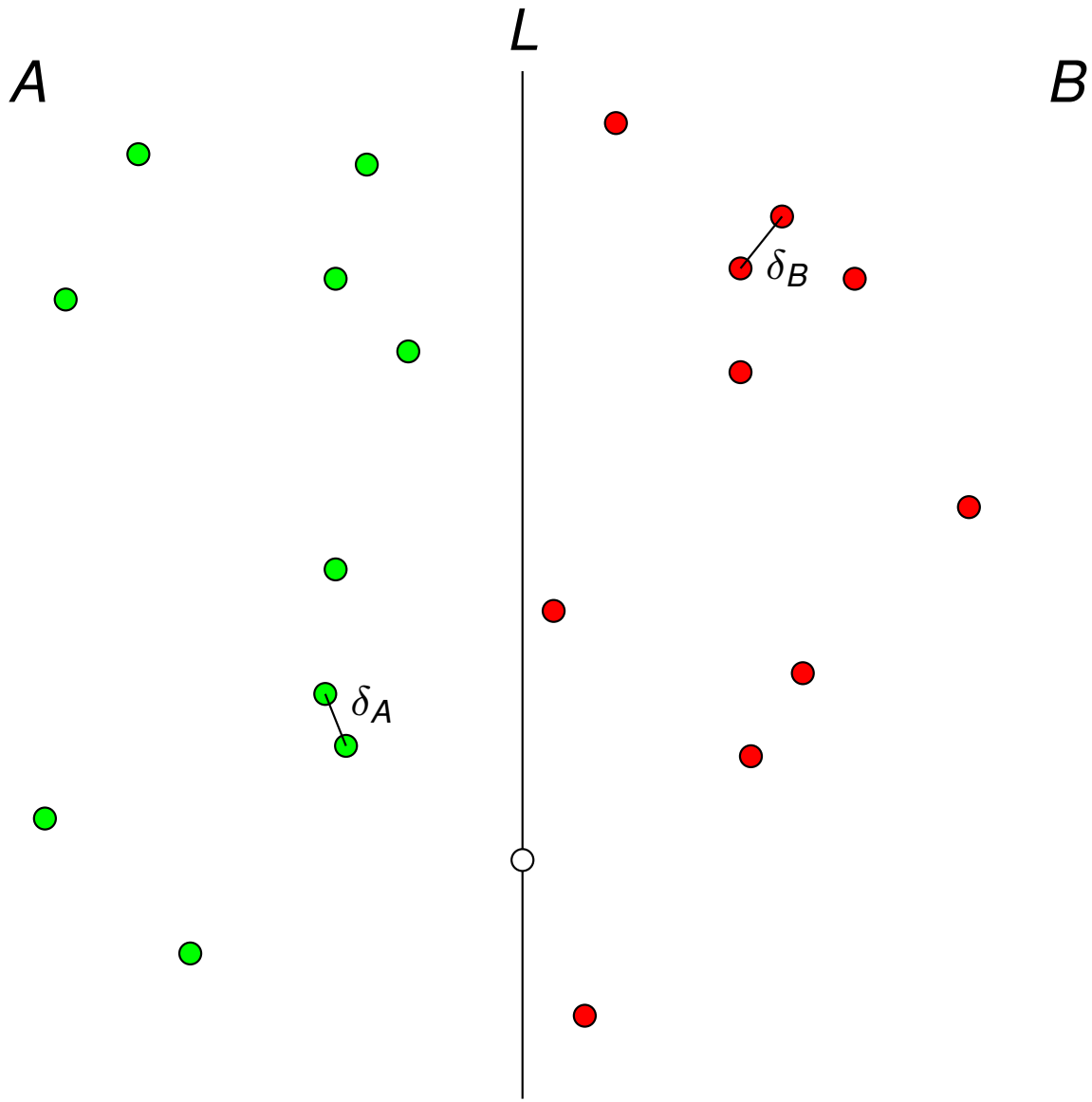
a configuration of 21 random points



split the points sorted by x-coordinate in the middle



find the closest pair in each half and compare minima



the beginning of a recursive algorithm

`Closest-Pair`(P)

Input: P, a list of points, first sorted on their x-coordinate,
points with same x-coordinate are sorted on y-coordinate.
Output: δ, the minimum distance between the closest pair in P,
(p, q), the closest pair of points in P.

$n := \#P$
if $n \leq 3$ then
return (δ, p, q), computed by comparing pairwise distances
else
let A contain the first $\lceil n/2 \rceil$ points of P
let B contain the remaining $\lfloor n/2 \rfloor$ points of P
$(\delta_A, p_A, q_A) :=$ `Closest-Pair`(A)
$(\delta_B, p_B, q_B) :=$ `Closest-Pair`(B)

What if there is a $p \in A$ and a $q \in B$ so that the distance between p and q is smaller than $\min(\delta_A, \delta_B)$?

Divide and Conquer

1. Divide and Conquer
   - obtaining subquadratic time
   - measuring disorder by counting inversions
   - a sort and count algorithm
2. Finding the Closest Pair of Points
   - distances between pairs of points in the plane
   - combining solutions of subproblems
3. Karatsuba's Integer Multiplication
   - improving the elementary school algorithm
   - a recursive integer multiplication

a property of the closest pairs

For P, a set of points sorted on the x-coordinate, with
δ_A the minimum distance for the first half of P, and
δ_B the minimum distance for the second half of P, let $\delta = \min(\delta_A, \delta_B)$.

Proposition (7)

If there is a $p \in A$ and a $q \in B$ for which the distance $d(p, q) < \delta$, then both p and q lie within a distance δ of the line L separating A and B.

Proof. $d(p, q) < \delta \Rightarrow |p_x - q_x| < \delta$. $p \in A$, $q \in B \Rightarrow 0 < q_x - p_x < \delta$.

If L_x is the x-coordinate of the separating line L, then $p_x \leq L_x \leq q_x$.

$L_x - p_x \leq q_x - p_x < \delta$ implies that p lies within distance δ from L.

$q_x - L_x \leq q_x - p_x < \delta$ implies that q lies within distance δ from L.

Q.E.D.

rephrasing the property

Given P, δ, and L_x, let $S = \left\{ (p_x, p_y) \in P \;\middle|\; |p_x - L_x| < \delta \right\}$.

S is the subset of P of all points within distance δ of L.

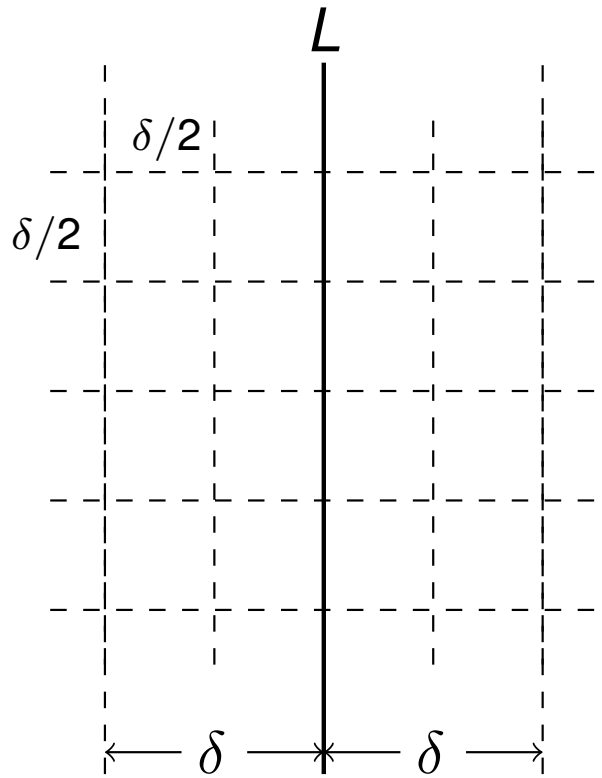
Corollary (8)

There exist a $p \in A$ and a $q \in B$ for which the distance $d(p, q) < \delta$ if and only if there exist $s, t \in S$ for which $d(s, t) < \delta$.

So we need to search through the set S,
sorted in increasing order of y-coordinate for the closest pair with $p \in A$ and $q \in B$.

Problem: what if S is just as large as P?

a partition in squares of sides of length $\delta/2$



Each square defines a box.

One row consists of 4 boxes.

Boxes on the same row have have the same y-coordinates.

If each box contains one point, then boxes determine positions.

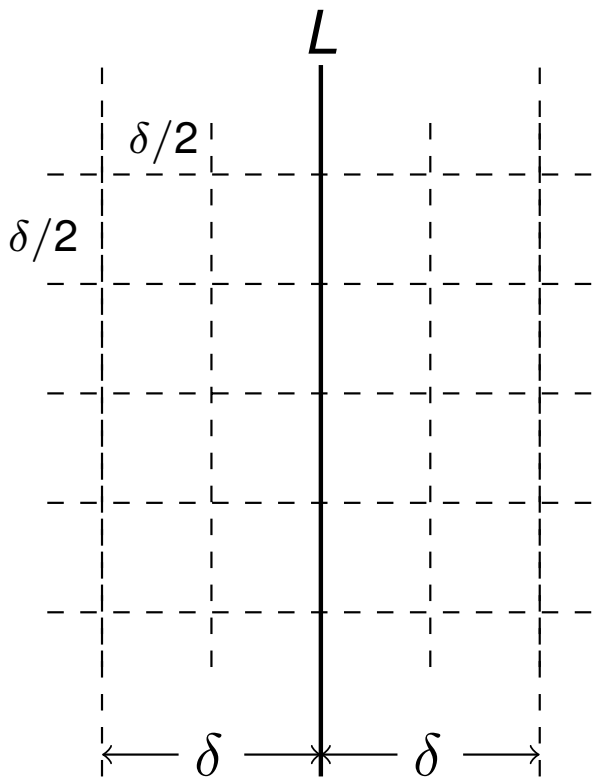
Theorem (9)

If $s, t \in S$ such that $d(s,t) < \delta$, then s and t are within 15 positions of each other in the list S_y, sorted on y-coordinate.

Theorem (9)

If $s, t \in S$ such that $d(s,t) < \delta$, then s and t are within 15 positions of each other in the list S_y, sorted on y-coordinate.

Proof.



If there would be two points in the same square, then their distance $\leq \delta\sqrt{2}/2 < \delta$. This contradicts $\delta = \min(\delta_A, \delta_B)$, so at most one point in each square.
Let $s, t \in S$, with $s_y < t_y$ and $d(s,t) < \delta$. By contradition, assume t is more than 15 positions father than s in the list S_y. Then, there are three rows of squares separating t from s and since the length of each square is $\delta/2$, the distance between t and s would be larger than $3\delta/2 > \delta$, which contradicts $d(s,t) < \delta$. Thus, s and t are within 15 positions of each other. Q.E.D.

points in 3-space

Exercise 4:

Consider the problem of computing the closest pair of n points in 3-space: $P = \{ (x, y, z) \mid x, y, z \in \mathbb{R} \}$.

Formulate a generalization of the theorem of the previous slide.

Prove the generalized version of the theorem.

computing the closest pair in S

`Closest-Pair-Combine`(P, L_x, δ)

Input: P, a list of points, first sorted on their x-coordinate, points with same x-coordinate are sorted on y-coordinate; L_x, the x-coordinate of the line which partitions P; $\delta = \min(\delta_A, \delta_B)$.

Output: δ_S, the minimum distance between the closest pair in S, (p_S, q_S), the closest pair of points in S.

$$S := \left\{ (p_x, p_y) \in P \;\middle|\; |p_x - L_x| < \delta \right\}$$

$\delta_S := \infty$

for $s \in S$ do

  for each $t \in S$, within the next 15 positions of s do

    $d := d(s, t)$, distance between s and t

    if $d < \delta_S$ then $\delta_S := d$; $p_S := s$; $q_S := t$

return (δ_S, p_S, q_S)

the algorithm is correct

Theorem (10)

Algorithm `Closest-Pair`*, jointly with* `Closest-Pair-Combine`*, correctly computes the closest pair of points.*

Proof. By induction on $n = \#P$. The base case is $n \leq 3$.
Comparing all distances between three points is correct.
Induction Hypothesis: the algorithm is correct for all P: $\#P = k < n$.
We have to prove that the algorithm is correct for P, with $\#P = n$.

- Applying the hypothesis on both halves of P:
  δ_A is the distance between the closest pair p_A, q_A,
  δ_B is the distance between the closest pair p_B, q_B.
- `Closest-Pair-Combine` returns δ_S, p_S, q_S and its correctness follows from Theorem (9).

So the minimum distance is $\min(\delta_A, \delta_B, \delta_S)$ between the corresponding pair of closest points. The algorithm is correct for P, $\#P = n$. Q.E.D.

cost of the combining the results

Lemma (11)

For $n = \#P$, `Closest-Pair-Combine` *runs in $O(n)$ time.*

Proof. Let $T(n)$ be the time of `Closest-Pair-Combine`.

As $S \subseteq P$, $\#S \leq n$.

The loop runs through all $s \in S$ and each step requires 15 $O(1)$ operations to compute the distance beween s and t.

For some $c > 0$: $T(n) \leq 15cn = O(n)$. Q.E.D.

cost of the algorithm

Theorem (12)

For $n = \#P$, the algorithm `Closest-Pair` *runs in $O(n \log(n))$ time.*

Proof. Let $T(n)$ be the time of `Closest-Pair` algorithm.

$T(n) \leq 2T(n/2) + c_1\, n$, for all $n \geq n_0$, for positive constants c_1 and n_0.

We have $T(2) \leq c_2$ and Theorem (5) applies. Q.E.D.

Divide and Conquer

1. Divide and Conquer
   - obtaining subquadratic time
   - measuring disorder by counting inversions
   - a sort and count algorithm
2. Finding the Closest Pair of Points
   - distances between pairs of points in the plane
   - combining solutions of subproblems
3. Karatsuba's Integer Multiplication
   - improving the elementary school algorithm
   - a recursive integer multiplication

CS 401/MCS 401 Lecture 8
Computer Algorithms I
Jan Verschelde, 6 July 2026

Divide and Conquer

1. Divide and Conquer
   - obtaining subquadratic time
   - measuring disorder by counting inversions
   - a sort and count algorithm
2. Finding the Closest Pair of Points
   - distances between pairs of points in the plane
   - combining solutions of subproblems
3. Karatsuba's Integer Multiplication
   - improving the elementary school algorithm
   - a recursive integer multiplication

integer multiplication

Input: $a_0, a_1, \ldots, a_{n-1}$, $a = a_{n-1}2^{n-1} + \cdots + a_1 2 + a_0$;
$b_0, b_1, \ldots, b_{n-1}$, $b = b_{n-1}2^{n-1} + \cdots + b_1 2 + b_0$.
Output: $c = a \times b$, as $c_0, c_1, \ldots, c_{2n-1}$.

$$
\begin{array}{cccccccc}
 & & & & & a_2 & a_1 & a_0 \\
 & & & & \times & b_2 & b_1 & b_0 \\
\hline
 & & & & a_2b_0 & a_1b_0 & a_0b_0 & \\
 & & & a_2b_1 & a_1b_1 & a_0b_1 & & \\
+ & & a_2b_2 & a_1b_2 & a_0b_2 & & & \\
\hline
 & c_5 & c_4 & c_3 & c_2 & c_1 & c_0 &
\end{array}
$$

Exercise 5: Write the elementary school algorithm for the multiplication of two n-bit integer numbers and show that it runs in $O(n^2)$.

applying divide and conquer

Let a and b be two n-bit integers.

1. We divide their bit sequences in half:

$$a = a_1 2^{n/2} + a_0, \quad b = b_1 2^{n/2} + b_0,$$

   where a_1, b_1 are the high order bits
   and a_0, b_0 are the bits of low order.

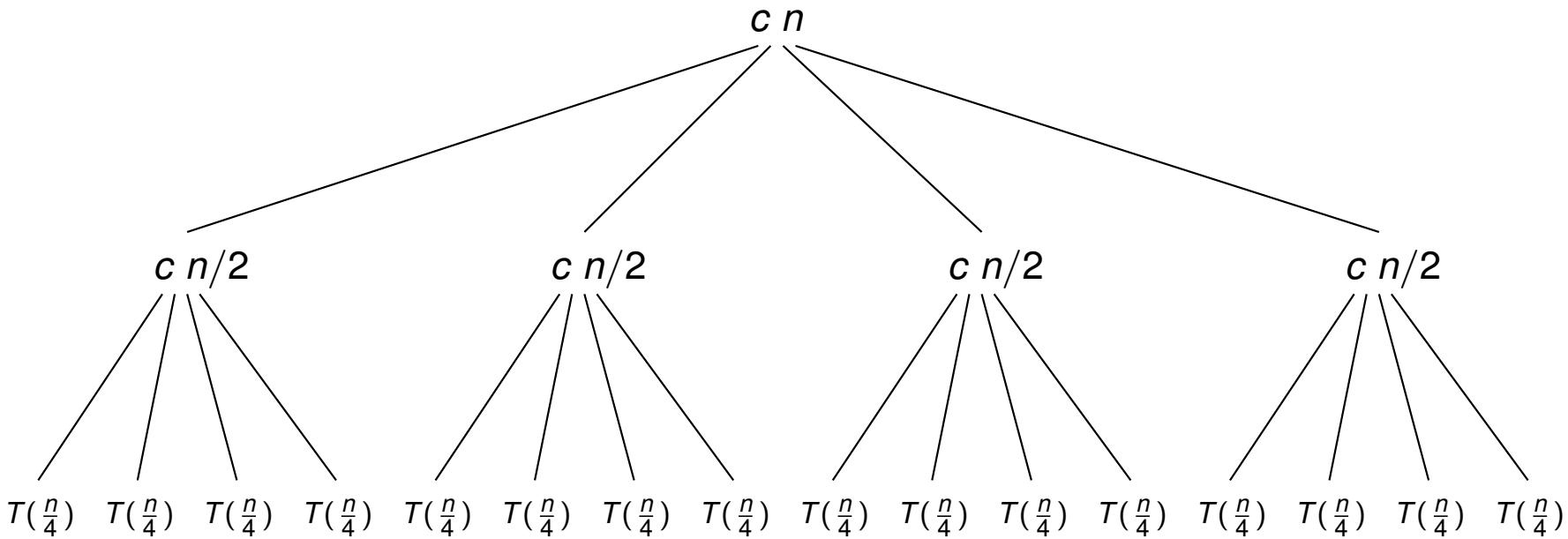
2. We multiply:

$$\begin{aligned} a \times b &= (a_1 2^{n/2} + a_0) \times (b_1 2^{n/2} + b_0) \\ &= a_1 \times b_1 2^n + (a_0 \times b_1 + a_1 \times b_0) 2^{n/2} + a_0 \times b_0. \end{aligned}$$

The multiplication of two n-bit numbers is replaced by

- four multiplications of numbers of size $n/2$, and
- three additions. Each addition costs $O(n)$.

unrolling the recurrence $T(n) \leq 4T(n/2) + cn$



We count the number of operations at each level:

0. cn
1. $cn/2 + cn/2 + cn/2 + cn/2 = 4(cn/2)$
2. $16(cn/4)$

There are $\log_2(n)$ levels: $T(n) \leq \sum_{\ell=0}^{\log_2(n)-1} \left(\frac{4}{2}\right)^{\ell} cn.$

computing a geometric sum

$$
\begin{aligned}
T(n) &\leq \sum_{\ell=0}^{\log_2(n)-1} \left(\frac{4}{2}\right)^{\ell} cn \\
&\leq cn \sum_{\ell=0}^{\log_2(n)-1} 2^{\ell} \\
&\leq cn\left(\frac{2^{\log_2(n)}-1}{2-1}\right) \\
&\leq cn(n-1) \\
&\text{is} \quad O(n^2).
\end{aligned}
$$

The straightforward divide and conquer yields no improvement over the elementary school integer multiplication algorithm.

Divide and Conquer

1. Divide and Conquer
   - obtaining subquadratic time
   - measuring disorder by counting inversions
   - a sort and count algorithm
2. Finding the Closest Pair of Points
   - distances between pairs of points in the plane
   - combining solutions of subproblems
3. Karatsuba's Integer Multiplication
   - improving the elementary school algorithm
   - a recursive integer multiplication

three recursive calls will do (Karatsuba)

For $a = a_1 2^{n/2} + a_0$ and $b = b_1 2^{n/2} + b_0$, consider

$$\begin{array}{rcccccccc} (a_1 + a_0)(b_1 + b_0) & = & a_1 b_1 & + & a_1 b_0 & + & a_0 b_1 & + & a_0 b_0 \\ & - & a_1 b_1 & & & & & & \\ & & & & & & & - & a_0 b_0 \\ \hline & & & & a_1 b_0 & + & a_0 b_1 & & \end{array}$$

Three recursive calls:

1. $p = (a_1 + a_0) \times (b_1 + b_0)$
2. $a_1 \times b_1$
3. $a_0 \times b_0$

Then $a \times b = a_1 b_1 2^n + (p - a_0 b_0 - a_1 b_1)2^{n/2} + a_0 b_0$.

Note: the length of $a_1 + a_0$ and $b_1 + b_0$ is $1 + n/2$ bits.

a recursive integer multiplication (Karatsuba)

assume: n is a power of 2

`Recursive-Multiply`(a, b, n)

Input: a, $a = a_1 2^{n/2} + a_0$ and b, $b = b_1 2^{n/2} + b_0$.
Output: c, $c = a \times b$, $c = c_2 2^n + c_1 2^{n/2} + c_0$.

if $n = 1$ then
    return $a \times b$
else
    $s_1 := a_1 + a_0$
    $s_0 := b_1 + b_0$
    $c_2 :=$ `Recursive-Multiply`$(a_1, b_1, n/2)$
    $c_0 :=$ `Recursive-Multiply`$(a_0, b_0, n/2)$
    $c_1 :=$ `Recursive-Multiply`$(s_1, s_0, n/2) - c_0 - c_2$
    return (c_2, c_1, c_0)

Karatsuba multiplication runs in subquadratic time

Theorem (13)

On two n-bit integers, `Recursive-Multiply` *runs in* $O(n^{1.59})$ *time.*

To prove this theorem, observe:

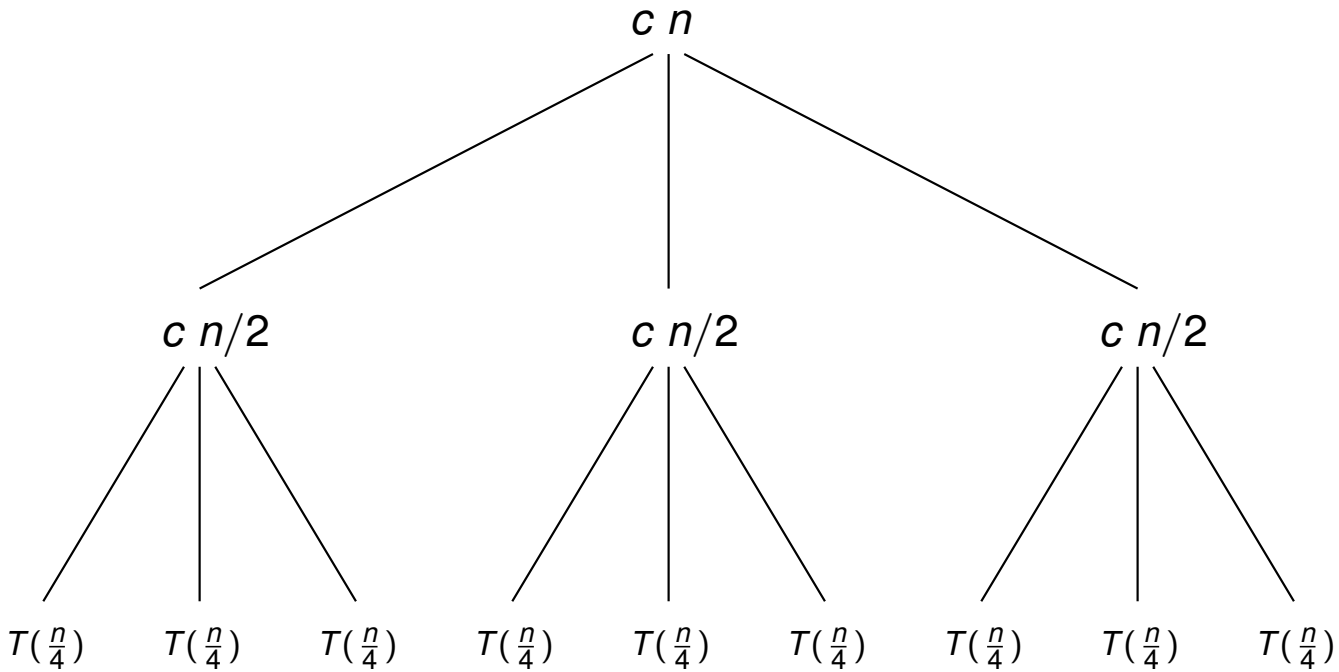
1. the algorithm makes three recursive calls, and
2. adding and subtracting numbers of n bits takes time $O(n)$.

The second observation is needed as a Lemma to prove the theorem.

Exercise 6:

1. Given two numbers a and b each of n bits long.
   Prove that computing $a + b$ takes $O(n)$ operations.
2. The `Recursive-Multiply` assumes n is a power of 2.
   However, s_0 and s_1 may have $1 + n/2$ bits.
   Show that fixing this bug in the algorithm adds $O(n)$ operations.

unrolling the recurrence $T(n) \leq 3T(n/2) + cn$



We count the number of operations at each level:

0. cn
1. $cn/2 + cn/2 + cn/2 = 3(cn/2)$
2. $9(cn/4)$

There are $\log_2(n)$ levels: $T(n) \leq \sum_{\ell=0}^{\log_2(n)-1} \left(\frac{3}{2}\right)^{\ell} cn.$

computing a geometric sum

$$\begin{aligned} T(n) &\leq \sum_{\ell=0}^{\log_2(n)-1} \left(\frac{3}{2}\right)^{\ell} cn \\ &\leq cn \sum_{\ell=0}^{\log_2(n)-1} \frac{3}{2}^{\ell} \\ &\leq cn\left(\frac{\frac{3}{2}^{\log_2(n)} - 1}{\frac{3}{2} - 1}\right) \end{aligned}$$

For $a > 1$ and $b > 1$, we have $a^{\log(b)} = b^{\log(a)}$, so $\frac{3}{2}^{\log_2(n)} = n^{\log_2(3/2)}$.

$$\log_2(3/2) = 0.58496250072115619, \quad T(n) \leq cnn^{0.59} \text{ is } O(n^{1.59}).$$

a comparison by example

Consider the multiplication of two multiprecision integers x and y of size 4. We have:

$$x = x_0 + x_1 B + x_2 B^2 + x_3 B^3 \text{ and } y = y_0 + y_1 B + y_2 B^2 + y_3 B^3,$$

where B is some power of 2, and $x_i, y_i \in [0, B-1]$, for $i = 0, 1, 2, 3$.

1. We first count the number of additions and multiplications to multiply x with y with the straightforward multiplication algorithm.
2. How many multiplications and additions (or subtractions) does the algorithm of Karatsuba require on numbers of size 4?

Which algorithm is expected to run faster?

running the straightforward algorithm

Multiplying x and y as integers of size 4:

$$\left(x_0 + x_1 B + x_2 B^2 + x_3 B^3\right) \times \left(y_0 + y_1 B + y_2 B^2 + y_3 B^3\right)$$

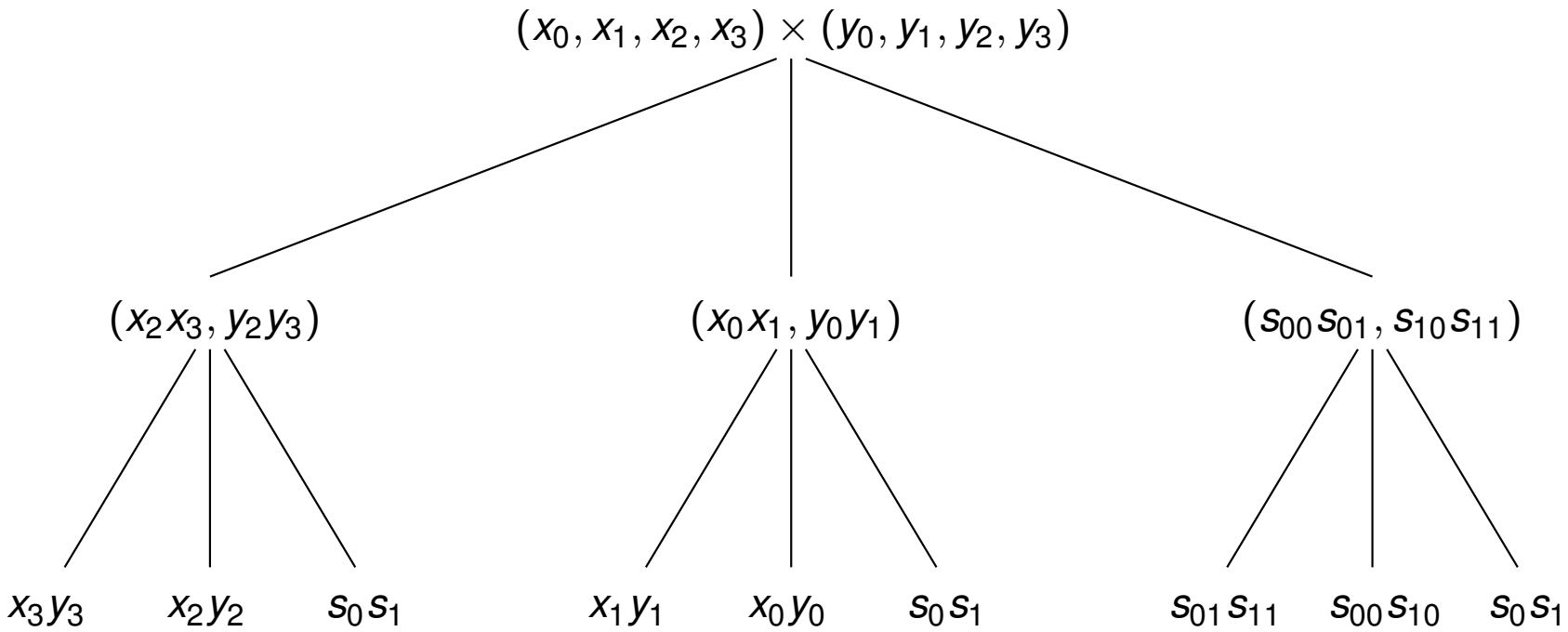
turns into

$$\begin{array}{llllllll}
x_0 \times y_0 & x_1 \times y_0 & x_2 \times y_0 & x_3 \times y_0 & & & & \\
 & +x_0 \times y_1 & +x_1 \times y_1 & +x_2 \times y_1 & +x_3 \times y_1 & & & \\
 & & +x_0 \times y_2 & +x_1 \times y_2 & +x_2 \times y_2 & +x_3 \times y_2 & & \\
 & & & +x_0 \times y_3 & +x_1 \times y_3 & +x_2 \times y_3 & +x_3 \times y_3 &
\end{array}$$

giving $z_0 + z_1 B + z_2 B^2 + z_3 B^3 + z_4 B^4 + z_5 B^5 + z_6 B^6 + z_7 B^7$,
where the z_i are the results of the above convolutions.

We count 16 multiplications and 9 additions.

running the Karatsuba integer multiplication



At the leaves we do 9 multiplications. At each recursive call we do 2 additions for s_0 and s_1 and then to subtract c_0 and c_2.
At the leaves, we have thus 3×4 additions (or subtractions).
At the inner nodes, we have also 4 additions (or subtractions), but then on double numbers which should be counted as 8.

In total, we count 20 additions (or subtractions).

comparing algorithms

Multiplying two integers of size 4, which algorithm is better?

1. The straightforward algorithm takes 16 multiplications and 9 additions.
2. The algorithm of Karatsuba takes 9 applications and 20 additions (which includes subtractions).

Assuming multiplications and additions have the same cost, the straightforward algorithm performs best.

For Karatsuba to be better than the straightforward algorithm, experiments with optimized software (NTL of Victor Shoup) have shown that Karatsuba becomes better for numbers of 500 bits.

Joachim von zur Gathen and Jürgen Gerhard:
Modern Computer Algebra, Cambridge University Press, 1999.